

Neural_Networks

Task	Convolutional	Transformer
Dense Prediction (segmentation & depth estimation)	FCN, U-Net, monoDepth	DPT, SETR
Image Recognition	ResNet-50, EfficientNet	ViT, DinoV2
Object Detection	YOLO, Faster R-CNN, CenterNet, FCOS	DETR
Image Captioning	CNN+LSTM,	CLIP

Supervised Learning

Self-Supervised Learning

RNN

1. RNN Cell

1. An RNN Cell forward

1. takes as input: x or h_{prev_l} , h_{prev_t}
2. produces: h_{next} and cache: $(h_{next}, h_{prev_l}, h_{prev_t}, dh_{next})$ (for backprop, in most implementations)

2. An RNN Cell backward

1. takes as input, upstream gradient dh_{next} , h_{next} , h_{prev_l} , h_{prev_t}
2. produces: dx or dh_{prev_l} , dh_{prev_t}

3. RNN Cell:

```
import numpy as np
from .activation import *

class RNNCell:

    def __init__(self, input_size, hidden_size):

        self.input_size = input_size
        self.hidden_size = hidden_size

        # Activation function for
        self.activation = Tanh()

        # hidden dimension and input dimension
        h = self.hidden_size
        d = self.input_size

        # Weights and biases
        self.W_ih = np.random.randn(h, d)
        self.W_hh = np.random.randn(h, h)
        self.b_ih = np.random.randn(h)
        self.b_hh = np.random.randn(h)

        self.dW_ih = np.zeros((h, d))
        self.dW_hh = np.zeros((h, h))
        self.db_ih = np.zeros(h)
        self.db_hh = np.zeros(h)

    def init_weights(self, W_ih, W_hh, b_ih, b_hh):
        self.W_ih = W_ih
        self.W_hh = W_hh
        self.b_ih = b_ih
        self.b_hh = b_hh
```

```

def zero_grad(self):
    h, d = self.hidden_size, self.input_size
    self.dW_ih = np.zeros((h, d))
    self.dW_hh = np.zeros((h, h))
    self.db_ih = np.zeros(h)
    self.db_hh = np.zeros(h)

def __call__(self, x, h_prev_t):
    return self.forward(x, h_prev_t)

def forward(self, x, h_prev_t):
    h_next = h_prev_t @ self.W_hh.T + self.b_ih + x @ self.W_ih.T + self.b_hh
    out = self.activation.forward(h_next)
    return out

def backward(self, dh_next, h_next, h_prev_l, h_prev_t):
    dz = self.activation.backward(dh_next, h_next)
    batch_size = dh_next.shape[0]
    # 1) Compute the averaged gradients of the weights and biases
    self.dW_ih += dz.T @ h_prev_l / batch_size
    self.dW_hh += dz.T @ h_prev_t / batch_size
    self.db_ih += np.sum(dz.T, axis=1) / batch_size
    self.db_hh += np.sum(dz.T, axis=1) / batch_size

    # 2) Compute dx, dh_prev_t
    dx = dz @ self.W_ih
    dh_prev_t = dz @ self.W_hh

    return dx, dh_prev_t

```

2. RNN Classifier

1. RNN Classifier forward:

```

def forward(self, x, h_0=None):
    """RNN forward, multiple layers, multiple time steps.
    x: input sequence: (N, T, D)
    self.x: cache (for backprop) of input sequence: (N, T, D)
    h0: initial hidden state (optional), zeros if not given: (L, N, H)
    self.hiddens: cache (for backprop) of computed hidden states (h_next): (T+1, L, N, H)
    hiddens: computed hidden states for all the layers (L, N, H)
    h_next: hidden state (N, H)
    logits: output (N, H_out)
    """
    N, T, D = x.shape
    L = self.num_layers
    H = self.hidden_size

    if h_0 is None:
        hiddens = np.zeros((L, N, H), dtype=np.float64)
    else:
        L, _, H = h_0.shape
        assert L == self.num_layers, H == self.hidden_size
        hiddens = h_0
    self.hiddens.append(hiddens.copy())
    self.x = x
    for t in range(T):
        for l in range(L):
            # Compute h_prev_l

```

```

        if l == 0:
            h_prev_l = x[:, t, :]
        else:
            h_prev_l = hiddens[l - 1, :, :]
        # Compute h_prev_t
        h_prev_t = hiddens[l]
        # Perform forward on RNN Cell
        h_next = self.rnn[l].forward(h_prev_l, h_prev_t)
        # Update hiddens for next time step
        hiddens[l] = h_next

    self.hiddens.append(hiddens.copy())

logits = self.output_layer.forward(h_next)
return logits

```

2. RNN Classifier backward,

1. BPTT for many-to-one, multiple layers
2. An important principle for BPTT: very simple rule: for each node with multiple descendants in the computational graph, simply add up the delta vectors coming from all of the descendants.

```

def backward(self, delta):
    """RNN Back Propagation Through Time (BPTT).
    N: number of samples or batch size
    L: number of layers
    H: hidden dimension
    D: input dimension
    H_out: output dimension
    delta: Upstream gradient, dL/dy from linear layer before loss function
    i.e. in last RNN Cell, h_next -> y -> L: (N, H_out)
    self.hiddens: Contains h_next computed in each RNN Cell: (T+1, L, N, H)
    self.x: input sequence: (N, T, D)
    dh_next: contains upstream gradients: (L, N, H)
    for backprop, cache = (h_next, h_prev_l, h_prev_t, dh_next)
    """

    T = self.x.shape[1]
    L = self.num_layers
    H = self.hidden_size
    N, _ = delta.shape
    dh_next = np.zeros((L, N, H), dtype=np.float64)
    # Compute upstream gradient, dL/dh_next, for last RNN Cell
    dh_next[-1] = self.output_layer.backward(delta)
    # self.output_layer.backward(delta)
    for t in reversed(range(T)):
        for l in reversed(range(L)):
            # Compute variables in cache need for backprop
            h_next = self.hiddens[t + 1][l, :, :]
            h_prev_t = self.hiddens[t][l, :, :]
            # h_prev_l can come from either input sequence or h_next from previous cell, l-1
            if l == 0:
                h_prev_l = self.x[:, t, :]
            else:
                h_prev_l = self.hiddens[t + 1][l - 1, :, :]
            # Done computing variables in cache needed for backprop
            # Perform backprop on RNN Cell
            h_prev_l, h_prev_t = self.rnn[l].backward(
                dh_next[l], h_next, h_prev_l, h_prev_t
            )
            # Update dh_next for the next previous time step

```

```

dh_next[1] = h_prev_t
# Add downstream gradient to the upstream gradient of the next previous cell step
if l != 0:
    dh_next[l - 1] += h_prev_l
return dh_next / N

```

LSTM

1. $\mathbf{i}_t = \sigma(\mathbf{W}_{ix}\mathbf{x}_t + \mathbf{W}_{ih}\mathbf{h}_{t-1})$,
2. $\mathbf{f}_t = \sigma(\mathbf{W}_{fx}\mathbf{x}_t + \mathbf{W}_{fh}\mathbf{h}_{t-1})$,
3. $\mathbf{o}_t = \sigma(\mathbf{W}_{ox}\mathbf{x}_t + \mathbf{W}_{oh}\mathbf{h}_{t-1})$,
4. $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_{cx}\mathbf{x}_t + \mathbf{W}_{ch}\mathbf{h}_{t-1})$,
5. $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$,

References:

1. [LSTM: A detailed Explanation](#), Towards Data Science, Rian Dolphin.

References:

1. Unsupervised Representation Learning by Predicting Image Rotations. Gidaris, et. al. Ecole des Ponts ParisTech.